

week4

Jiajun Li

2026-06-22

Answers to the questions from recent emails

Whether to expand the research scope

I plan to allocate my time towards a more detailed explanation of existing framework. This includes detailing the specific reasons for data processing steps and improving the interpretation of the SHAP values.

Missing data imputation

I carried out a brief experiment after setting up the model. Moving forward, I will use the 'NA' string imputation strategy proposed by Ryan et al.

What I did

week3

- Completed all path-related work. Codes and the datasets used are located in the `python_codes` folder.
- All data has been fully integrated.

week4

- Introduction of final report.
- Completed with the data cleaning, preprocessing, and data splittings tasks.
- Finished the definition of cross validation and evaluation of models.
- A small experiment: Compare the results of leaving structural missing values as `np.nan` against filling them with an “N/A” string category, or just remove these columns.

Experiment method

This section illustrates the main process of three different version.
Full implementation details are available in the `python_code` folder.

String Version

```
model_df_str = model_df.copy()
```

```
model_df_str[exp_cols] = model_df_str[exp_cols].fillna("N/A")
```

#NaN Version

```
model_df_nan = model_df.copy()
```

#Drop version

```
model_df_drop = model_df.dropna(subset=exp_cols).copy()
```

For the `model_df` dataset, all systematic missing values within the time period have already been converted to null values.

This section illustrates the main calling sequence for splitting the data. Full implementation details are available in the `python_code` folder.

```
split_and_save(processed_nan, "exp_nan")  
split_and_save(processed_str, "exp_str")  
split_and_save(processed_drop, "exp_drop")
```

Gridsearch settings

```
grid_xgb = {  
    'learning_rate': [0.05, 0.1, 0.2],  
    'n_estimators': [100, 250, 500],  
    'max_depth': [4, 6, 8],  
    'colsample_bytree': [0.6, 0.8, 1.0]  
}  
xgb_clf = XGBClassifier(  
    eval_metric='logloss',  
    random_state=SEED,  
    enable_categorical=True  
)
```

Results

- Based on the results of the experiment, imputing 'NA' string did not affect the model's predictive performance, and removing specific features make changes to the predictions.
- Because most models to be evaluated cannot directly handle np.nan values, I will proceed with the string imputation strategy .

Results

exp_nan: {'colsample_bytree': 0.6, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 250} Mean ROC AUC of cv:0.898 (0.002)

exp_str: {'colsample_bytree': 0.6, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 500} Mean ROC AUC of cv:0.898 (0.002)

exp_drop: {'colsample_bytree': 0.6, 'learning_rate': 0.05, 'max_depth': 8, 'n_estimators': 500} Mean ROC AUC of cv:0.899 (0.002)

Plan for next week

- Model tuning.
- Begin my presentation slides
- Keep working on my final report

Model tuning (w3-6)

Fit four models: Logistic Regression, Classification Trees, XGBoost, and MLPs. For the tree based models, hyperparameters will be tuned with 5-fold cross validation on the training data, with AUC as the optimisation metric.

Model comparison (w6-7)

Compare the four models using AUC, Precision, Recall, Accuracy, and F1 score. The best models will be selected based on predictive performance.

Interpretation (w7-9/10)

Investigate the stable important features by SHAP values on the two best performing models.